

SQL Query Writing

Exam cheatsheet: relational algebra equivalences, methods, traps, and solved querying exercise

SQL is close to relational algebra, but not identical. Relational algebra uses set semantics by default, while SQL uses bag semantics unless `DISTINCT` or a set operator removes duplicates. In the exam, write the simplest correct query, make joins explicit, and state when `DISTINCT` is needed.

SQL Skeleton And Logical Order

The written order and the evaluation order are different. This matters for `WHERE`, `GROUP BY`, `HAVING`, and aliases.

Clause	Role	Exam rule
FROM and JOIN	Build the input relation by combining tables.	Put relationship conditions in <code>ON</code> or <code>USING</code> ; qualify attributes when names repeat.
WHERE	Filter individual rows before grouping.	Use for normal predicates such as <code>stars >= 4</code> , <code>Name = 'Peter'</code> , or join-independent filters.
GROUP BY	Partition rows into groups.	Every non-aggregate expression in <code>SELECT</code> must appear in <code>GROUP BY</code> .
HAVING	Filter groups after aggregation.	Use for aggregate predicates such as <code>COUNT(*) > 1</code> or <code>AVG(Rating) = ...</code> .
SELECT	Choose output expressions.	Use aggregate functions only with the correct grouping rule.
DISTINCT	Remove duplicate output rows.	Needed when SQL bag semantics can output duplicates but the relational answer is a set.
ORDER BY	Sort final result.	Usually not needed unless the question asks for an order.

The practical order to design a query is different again: decide the output attributes, identify source tables, write joins, add filters, decide whether grouping is needed, decide whether duplicates can appear, then simplify.

Relational Algebra To SQL Equivalences

Relational algebra	SQL pattern	Important difference
$R[A,B]$	<code>SELECT DISTINCT A, B FROM R</code>	Use <code>DISTINCT</code> if you want set semantics. Without it, SQL may keep duplicates.
$R:P$	<code>SELECT * FROM R WHERE P</code>	Rows where P is false or unknown are discarded. This matters with <code>NULL</code> .
$R * S$	<code>FROM R JOIN S USING (common_attribute)</code>	Natural join in RA joins on all common names. In SQL, <code>USING</code> joins only listed names.
$R(P)*S$	<code>FROM R JOIN S ON (P)</code>	Use <code>ON</code> when names differ or the condition is not just equality on same names.
$R \times S$	<code>FROM R CROSS JOIN S</code>	Rarely useful alone; most queries need a join condition.
$T(X,Y) \leftarrow R$	<code>FROM R AS T</code> or <code>SELECT A AS X</code>	Aliases are mandatory for self-joins.
$R \cup S$	<code>query1 UNION query2</code>	<code>UNION</code> removes duplicates; <code>UNION ALL</code> keeps them.
$R \cap S$	<code>query1 INTERSECT query2</code>	Inputs must have compatible schemas.
$R - S$	<code>query1 EXCEPT query2</code> or Oracle <code>MINUS</code>	Set difference is the cleanest way to express “never” or “not in” when schemas match.
R / S	<code>GROUP BY ... HAVING COUNT(DISTINCT ...) = ...</code> or double <code>NOT EXISTS</code>	SQL has no direct division operator.
Aggregation in extended RA	<code>GROUP BY, COUNT, AVG, MIN, MAX, SUM</code>	Basic RA from the slides does not express grouping or averages.

Query Design Method

Step	Action
1	Underline the requested output. If the question asks for titles and authors, the final <code>SELECT</code> should contain only those attributes unless extra attributes are explicitly requested.
2	List the tables needed to connect the output to the condition. If the condition mentions reader names and the output is book data, the path is <code>READER -> WISHLIST -> BOOK</code> .
3	Write explicit joins. Prefer <code>JOIN ... ON</code> when attribute names differ or ambiguity is possible. <code>USING</code> is fine when the shared attribute name is exactly the intended join key.
4	Add row filters in <code>WHERE</code> . These are predicates that apply before aggregation.
5	If the question asks for “number”, “average”, “highest average”, “more than once”, or “for each”, decide the grouping key and write <code>GROUP BY</code> .
6	If the question asks for absence, use <code>EXCEPT</code> , <code>NOT EXISTS</code> , or <code>LEFT JOIN ... IS NULL</code> . Avoid <code>NOT IN</code> when the subquery could contain <code>NULL</code> .
7	If the question asks for “all”, use division logic: compare counts of distinct required items, or use double <code>NOT EXISTS</code> .
8	Check duplicates. Add <code>DISTINCT</code> only when the selected attributes can repeat and the expected answer is a set.

Core Templates

Problem type	Template
Simple join	<code>SELECT DISTINCT wanted_columns FROM A JOIN B ON A.key = B.key WHERE row_condition;</code>
Grouped count	<code>SELECT group_attribute, COUNT(*) AS n FROM R WHERE row_condition GROUP BY group_attribute HAVING COUNT(*) > threshold;</code>
Count distinct values	<code>SELECT group_attribute, COUNT(DISTINCT value_attribute) AS n FROM R GROUP BY group_attribute;</code>
Maximum aggregate with ties	<code>WITH Scores AS (SELECT x, AVG(y) AS avg_y FROM R GROUP BY x</code>

Problem type	Template
	<pre>), MaxScore AS (SELECT MAX(avg_y) AS max_avg_y FROM Scores) SELECT x FROM Scores JOIN MaxScore ON avg_y = max_avg_y; </pre>
Absence with set difference	<pre> SELECT key FROM Candidate EXCEPT SELECT key FROM Bad; -- Oracle uses MINUS instead of EXCEPT. </pre>
Absence with NOT EXISTS	<pre> SELECT c.key FROM Candidate c WHERE NOT EXISTS (SELECT 1 FROM Bad b WHERE b.key = c.key); </pre>
Include rows with no match	<pre> SELECT a.key, b.value FROM A a LEFT JOIN B b ON b.key = a.key; </pre>
Find rows with no match	<pre> SELECT a.key FROM A a LEFT JOIN B b ON b.key = a.key WHERE b.key IS NULL; </pre>
Self-join for pairs	<pre> SELECT DISTINCT r1.target FROM R r1 JOIN R r2 ON r1.target = r2.target AND r1.witness < r2.witness; </pre>
All required items by counting	<pre> WITH Required AS (SELECT DISTINCT item FROM S), Have AS (SELECT candidate, COUNT(DISTINCT item) AS n FROM R WHERE item IN (SELECT item FROM Required) GROUP BY candidate), Need AS (SELECT COUNT(*) AS n FROM Required) SELECT candidate FROM Have JOIN Need USING (n); </pre>

Duplicates And DISTINCT

SQL duplicates are the biggest difference from relational algebra. The exam explicitly asks you to mark when DISTINCT is needed.

Case	Decision
Selecting a primary key from its own table	DISTINCT is usually unnecessary because the key is already unique.
Selecting non-key attributes after a join	DISTINCT is often needed, because several joined rows may project to the same visible result.
Selecting titles and authors instead of ISBNs	DISTINCT may be needed because different ISBNs can share the same title and author.
Using GROUP BY on exactly the output attributes	DISTINCT is unnecessary because grouping already creates one row per group.
Using UNION, INTERSECT, EXCEPT, or MINUS	DISTINCT is unnecessary for the set operator result because duplicates are removed by default.
Using UNION ALL	Duplicates are intentionally kept. Do not use it unless multiplicities matter.

Aggregation Rules

Rule	Consequence
COUNT(*)	Counts rows, including rows where some attributes are NULL.
COUNT(A)	Counts non-NULL values of A.
COUNT(DISTINCT A)	Counts different non-NULL values of A. Use this for “number of authors” if an author can have several books.
AVG, SUM, MIN, MAX	Ignore NULL values. If all values are NULL, the result is NULL.
WHERE before GROUP BY	Filters rows before aggregation. Example: keep only ratings of books from one author.
HAVING after GROUP BY	Filters groups after aggregation. Example: keep books with COUNT(*) > 1.
Non-aggregate in SELECT	Must appear in GROUP BY. If you select Year, Author, COUNT(*), then group by Year, Author.
Joining before counting	A join may multiply rows. Use COUNT(DISTINCT ...) when counting entities rather than joined rows.

Absence, Negation, And NULL

Trap	Safe method
NOT IN with possible NULL	Avoid it. If the subquery contains NULL, SQL three-valued logic can make the result empty or surprising. Use NOT EXISTS instead.
Using <> to find things that do not exist	Wrong for absence. Inequality joins compare existing pairs; they do not prove that no matching row exists. Use EXCEPT or NOT EXISTS.
Filtering right table after LEFT JOIN	A predicate on the right table in WHERE can turn the outer join into an inner join. Put match conditions in ON; put right.key IS NULL in WHERE only when looking for missing matches.
NULL = NULL	This is unknown, not true. Use IS NULL or IS NOT NULL.

Join Choices

Join form	Use
JOIN ... ON	Best default. It is explicit, handles different attribute names, and avoids accidental joins on same-name attributes.
JOIN ... USING (A)	Good when both tables have the same join column name and that is exactly the intended equality condition.
NATURAL JOIN	Avoid in exams unless explicitly requested. It joins on all same-name attributes, which can silently change if schemas change.

Join form	Use
CROSS JOIN	Use only when the Cartesian product is intended, such as constructing all candidate-required pairs for division logic.
LEFT JOIN	Use when rows from the left table must appear even if there is no matching right row.
Self-join	Use aliases such as R r1 and R r2. Add an anti-symmetry condition like r1.id < r2.id when unordered pairs are enough.

Exam Exercise 3: Subject

Database schema:

```

READER(RId, Name, YoB)
BOOK(ISBN, Title, Author, Year)
WISHLIST(RId, ISBN)
RATING(RId, ISBN, Rating)

```

The intended meanings are: READER stores reader identifiers, names, and years of birth; BOOK stores books identified by ISBN; WISHLIST stores which reader has which book in their wishlist; RATING stores ratings from 1 to 5. Only one rating per reader and per book is allowed.

Queries to write in SQL syntax, and where possible in relational algebra, using as few subqueries as possible and clearly marking when DISTINCT is needed:

Part	Request
a	The book titles and authors in the wishlist of the reader named Peter.
b	The books, by ISBN, that have been added to a wishlist but never rated.
c	The books, by title and author, that have been rated more than once.
d	The names of authors having the highest average rating.
e	The names of authors which have had none of their books rated.
f	List the number of authors for each year, for years in which at least one author has published a book.

Exercise 3: Solutions

a) Titles and authors in Peter's wishlist

```

SELECT DISTINCT b.Title, b.Author
FROM READER r
JOIN WISHLIST w ON w.RId = r.RId
JOIN BOOK b ON b.ISBN = w.ISBN
WHERE r.Name = 'Peter';

```

DISTINCT is needed because Name is not declared as a key, so there may be several readers named Peter, and because different ISBNs can project to the same (Title, Author) pair.

Relational algebra:

```
(READER:Name='Peter' * WISHLIST * BOOK)[Title, Author]
```

If using theta joins instead of natural joins, explicitly join READER.RId = WISHLIST.RId and WISHLIST.ISBN = BOOK.ISBN.

b) ISBNs in a wishlist but never rated

```

SELECT ISBN
FROM WISHLIST
EXCEPT
SELECT ISBN
FROM RATING;

```

Use MINUS instead of EXCEPT in Oracle-style SQL. DISTINCT is not needed because EXCEPT and MINUS return a set.

Alternative without set difference:

```

SELECT DISTINCT w.ISBN
FROM WISHLIST w
WHERE NOT EXISTS (
    SELECT 1
    FROM RATING r
    WHERE r.ISBN = w.ISBN
);

```

Relational algebra:

```
WISHLIST[ISBN] - RATING[ISBN]
```

c) Books rated more than once

```

SELECT DISTINCT b.Title, b.Author
FROM BOOK b
JOIN RATING r1 ON r1.ISBN = b.ISBN
JOIN RATING r2 ON r2.ISBN = r1.ISBN
AND r1.RId < r2.RId;

```

The self-join finds two different readers who rated the same ISBN. The condition r1.RId < r2.RId avoids symmetric duplicate pairs. DISTINCT is still needed because a book with many ratings creates several pairs, and because several ISBNs may share the same title and author.

Equivalent grouped solution:

```

WITH RatedMoreThanOnce AS (
    SELECT ISBN
    FROM RATING
    GROUP BY ISBN
    HAVING COUNT(*) > 1
)
SELECT DISTINCT b.Title, b.Author
FROM BOOK b
JOIN RatedMoreThanOnce r ON r.ISBN = b.ISBN;

```

Relational algebra:

```
R1(r1, isbn1, rating1) <- RATING[RId, ISBN, Rating]
R2(r2, isbn2, rating2) <- RATING[RId, ISBN, Rating]
ManyISBN(isbn) <- (R1(R1.isbn1 = R2.isbn2 and R1.r1 <> R2.r2)*R2)[isbn1]
Result <- (ManyISBN * BOOK)[Title, Author]
```

d) Authors with the highest average rating

```
WITH AuthorAverage AS (
  SELECT b.Author, AVG(r.Rating) AS avg_rating
  FROM BOOK b
  JOIN RATING r ON r.ISBN = b.ISBN
  GROUP BY b.Author
), MaxAverage AS (
  SELECT MAX(avg_rating) AS max_avg_rating
  FROM AuthorAverage
)
SELECT Author
FROM AuthorAverage
JOIN MaxAverage ON avg_rating = max_avg_rating;
```

DISTINCT is not needed because AuthorAverage has one row per author. Ties are preserved because every author whose average equals the maximum is returned. Authors with no ratings are not candidates for highest average rating because they have no average rating.

Basic relational algebra from the slides cannot express this query because it requires AVG, GROUP BY, and MAX.

e) Authors with none of their books rated

```
SELECT Author
FROM BOOK
EXCEPT
SELECT b.Author
FROM BOOK b
JOIN RATING r ON r.ISBN = b.ISBN;
```

Use MINUS instead of EXCEPT in Oracle-style SQL. DISTINCT is not needed because the set operator removes duplicates.

Alternative with NOT EXISTS:

```
SELECT DISTINCT b.Author
FROM BOOK b
WHERE NOT EXISTS (
  SELECT 1
  FROM BOOK b2
  JOIN RATING r ON r.ISBN = b2.ISBN
  WHERE b2.Author = b.Author
);
```

Relational algebra:

```
BOOK[Author] - (BOOK * RATING)[Author]
```

f) Number of authors for each publication year

```
SELECT Year, COUNT(DISTINCT Author) AS author_count
FROM BOOK
GROUP BY Year;
```

COUNT(DISTINCT Author) is needed because the same author can publish several books in the same year. No HAVING clause is needed: grouping over BOOK already lists only years in which at least one book exists. Basic relational algebra from the slides cannot express this query because it requires grouping and counting.

Final Exam Checklist

Before submitting	Check
Output columns	They exactly match the requested attributes. Extra IDs are not shown unless asked.
Join path	Every table is connected by a meaningful key equality. No accidental Cartesian product remains.
Duplicates	DISTINCT is present only when projection can duplicate visible rows.
Absence	Uses EXCEPT, MINUS, NOT EXISTS, or LEFT JOIN ... IS NULL, not an inequality join.
Aggregates	Every non-aggregate selected expression is in GROUP BY; aggregate filters use HAVING.
Counts	Uses COUNT(DISTINCT ...) when counting entities rather than rows.
Ties	Maximum or minimum queries return all tied answers unless the question says only one.
Relational algebra	Provided only where basic RA can express the query; aggregation queries are marked as requiring extended RA or SQL.